

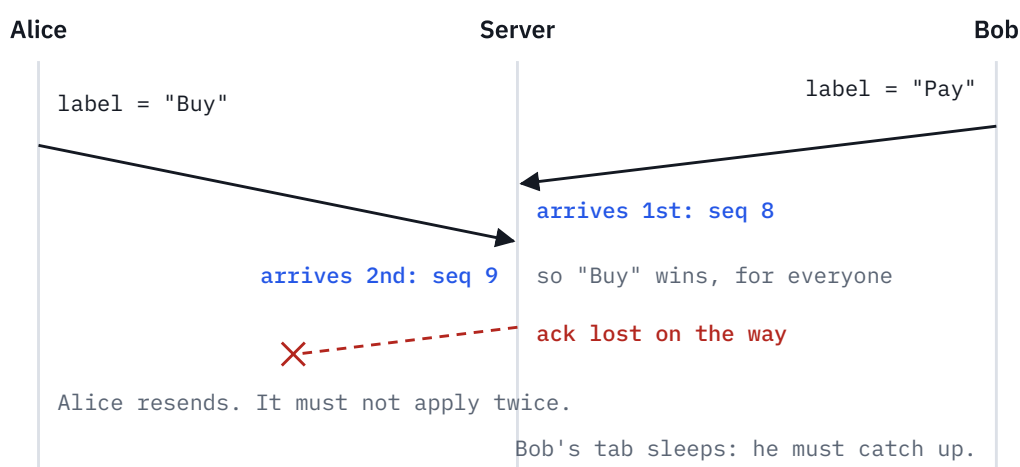
Many editors, one document: the approaches, and why we picked ours

A whiteboard-ready map. First the problems every multi-user editor must solve. Then the families of answers, with honest costs. Then what this project does, and the one requirement that decided it.

1. The problem

Alice and Bob have the same page open. It holds one Button. Alice sets its `label` to "Buy". At the same moment Bob sets it to "Pay".

Each sees their own edit at once. The network can delay, repeat, reorder or lose their messages. A laptop can sleep. A server can die. Yet both screens must end up showing the same label.



Any design, whatever its name, must answer these nine questions. Write this list on the whiteboard first.

1. **Order.** Two edits happened "at the same time". Which one counts as later?
2. **Convergence.** Does everyone end up with the same document, always?
3. **Instant feel.** My own edit must show now, not after a round trip.
4. **Duplicates and retries.** A resend must change the document once, not twice (exactly-once *effect*).
5. **Reconnect.** After a dropped socket, how do I catch up, and what happens to my unsent edits?
6. **Conflicting intent.** Edit a node that someone deleted. Move a frame into its own child (a cycle). Two inserts at one index.

7. **Validity and permission.** Who says an edit is allowed at all?
8. **Overload and abuse.** One client sends 10,000 edits a second.
9. **Durability and failover.** The server dies. What is lost? Who owns the document next?

2. The families of answers

Six ideas are worth knowing. The first two avoid the problem. The next three solve it. The last is a different layer that people mix in by mistake.

APPROACH	WHO DECIDES ORDER	OFFLINE / PEER-TO-PEER	ENFORCING VALIDATION, PERMISSIONS	MOVES IN A TREE	METADATA COST	BUILD COST
A. Lock / check-out	Nobody: one editor at a time	No	Easy, at the lock holder	Trivial	None	Low
B. Last write wins, whole document	The clock or the last save	Sort of, but work is lost	Easy, at save	Trivial	None	Lowest
C. Operational Transformation	Central server; ops are rewritten to fit	Short offline only, in practice	Possible: a server is in the path	Hard: transform rules multiply	Low	High
D. True CRDT (Yjs, Automerge)	Nobody: the merge rule is in the data	Yes, this is the point	Hard: no one is the authority	Hard: cycles need a special algorithm	Grows with edit history	Low with a library, high without
E. Server-ordered op log (ours, Figma-style)	One server process per document	Brief offline, then replay; no p2p	Natural: every op passes one gate	Simple: the server rejects cycles	A counter per op	Medium
F. Raft / replicated state machine	A different layer. It makes the <i>server's</i> log survive machine failures. It says nothing about how browsers merge edits. You can put it under C or E.					

A. Pessimistic locking, or check-out

Only one person may edit an object at a time. Everyone else waits or reads. This is `SELECT ... FOR UPDATE` held by a human.

Correct and simple, but people block each other, and a lock held by a closed laptop needs a timeout. It still suits rare, heavy edits: a contract, a binary asset.

B. Last write wins on the whole document

Every save sends the full document. The last save replaces everything. Alice fixes a label, Bob moves an image, and Bob's save erases Alice's fix.

Add a version column and the second writer gets a conflict error instead: classic optimistic locking. Honest, but now a human merges by hand. Fine for a settings form, not for a canvas.

C. Operational Transformation (OT)

Clients send small ops such as "insert 'x' at position 5". A central server puts them in order. When two ops were made at the same time, one is *transformed* so it still means the same thing after the other. Insert at 5 becomes insert at 6.

Google Docs is the well-known user. OT is excellent for text, where positions shift all the time. Its cost is the transform function: one for every pair of op types, each exactly right. For a tree with add, move, remove and set, the pairs multiply.

D. True CRDTs

A CRDT is a data type with a merge rule built in. Any two copies that have seen the same edits, in any order, hold the same state. No server has to decide anything. Yjs and Automerge are the two best-known libraries.

This is the right tool when there is no server you can count on: offline for days, peer-to-peer, local-first apps. You pay for that freedom in four ways:

- **Metadata.** Every item carries ids, and deletes usually leave a marker (a tombstone) or stay in the history. Size follows the edit history, not only the content.
- **Tree moves are hard.** Alice moves A into B while Bob moves B into A. Each move is fine alone; merged, they make a cycle. Solving this needed its own research paper (Kleppmann and others, 2021).
- **No authority.** If nobody decides, nobody can refuse. Schema validation, permissions, rate limits and "who did this" must be bolted on by a server anyway, and a server that rejects edits breaks the "any order converges" promise.
- **Converged is not the same as intended.** The merge is always consistent. It is not always what either user meant.

E. Server-authoritative ordered op log

Clients send small ops. One server process per document gives each accepted op the next number. Everyone applies ops in that number order. Same ops, same order, same result: convergence without a merge function.

In Java terms: the room is a Kafka partition with a single writer. The sequence number is the offset. Every replica is a consumer that replays the log. Concurrent edits to one prop resolve as last writer wins, where "last" means "highest number", never a clock.

F. Raft and replicated state machines

Raft keeps one log identical on three or five *servers*, so a machine can die and the log survives. It is what etcd and Consul use inside.

Saying "use Raft" for collaborative editing confuses two layers. Raft answers problem 9, the server's own durability and failover. It does nothing for problems 1 to 8 between browsers. Our planned answer to problem 9 is simpler: Postgres holds the truth, and a lease with a fencing token decides who may write (section 4).

3. What Figma chose, and why

A correction worth getting right in an interview. Figma did not adopt CRDTs and later drop them. Their 2019 engineering post says they looked at OT and at CRDTs, and built something simpler that borrows CRDT ideas. In their words: "Figma isn't using true CRDTs though."

Why not OT. The post's verdict: "OTs were unnecessarily complex for our problem space." A design file is a tree of objects with properties, not a long string of characters.

Why not a true CRDT. CRDTs are built for systems with no central authority, and that costs overhead. Figma has a server. Again their words: "Since Figma is centralized (our server is the central authority), we can simplify our system by removing this extra overhead and benefit from a faster and leaner implementation."

What they built, as described in that post and its 2017 companion:

- **Last writer wins per property, per object.** The document is a map of objects, each a map of properties. Two people change *different* properties of one object: both changes stay. They change the *same* property: the value that reached the server last wins. The server defines the order.
- **No flicker.** While a client has an unacknowledged change to a property, it ignores incoming server values for that same property. Its own change is its best guess of the final value.
- **Tree structure.** The parent link is a property stored on the child. If a re-parent would create a cycle, the server rejects it.
- **Child order by fractional indexing.** Each child has a position between 0 and 1. To insert between two children, take a position between theirs. No other child has to be renumbered. If two clients pick the same position, the server assigns a unique one to the second. Concurrent inserts may interleave, which is acceptable for design objects.
- **An honest limit.** Text is a single property, so two people typing in the same text box do not merge. The post says so plainly.

TINY EXAMPLE: LWW PER PROPERTY

Alice sets `Button.label = "Buy"` . Bob sets `Button.color = "red"` . Result: a red "Buy" button. No conflict, because the unit of conflict is one property, not the object and not the document. Now both set `label` : the one the server saw last wins, on every screen.

The lesson to carry: **if a server is always in the path, let it decide the order, and you can delete most of the hard parts of a CRDT.**

4. What we do: the life of one op

This project is family E. SPEC.md states two keystones. "One room, one order": each document has exactly one in-memory room, the only sequencer; client clocks are never used. "One write path": browser, AI agent and git peer all submit ops through `packages/peer-client` .

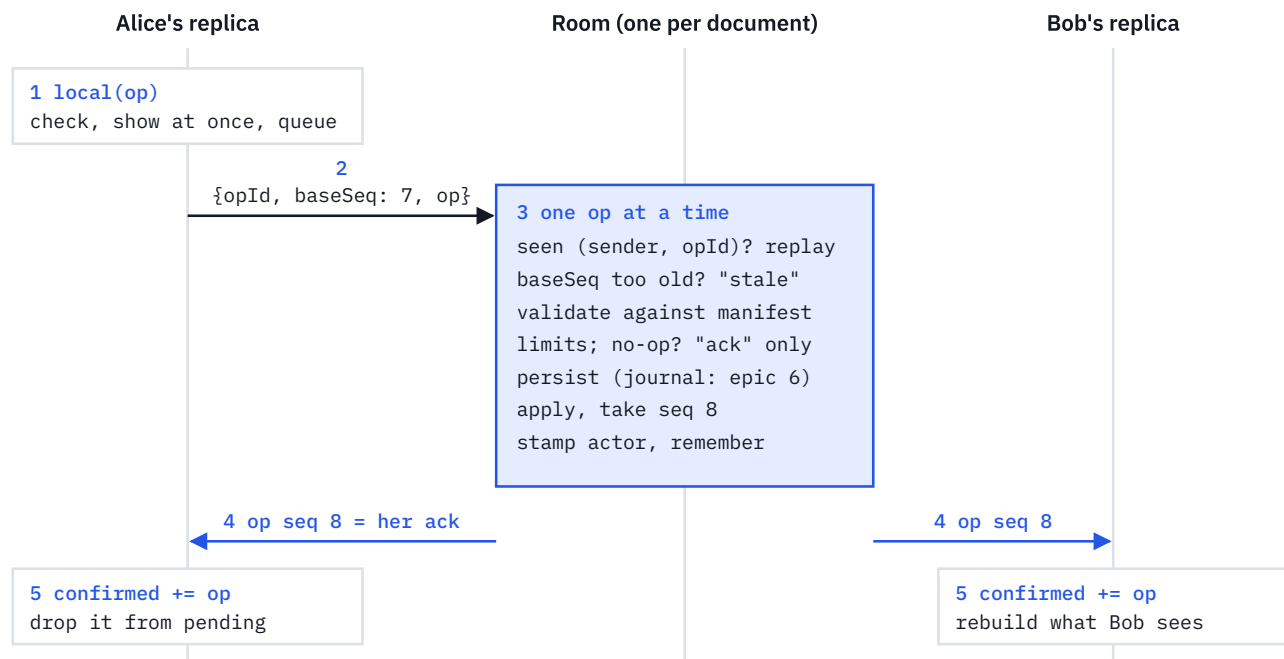
The whole vocabulary of change is four ops: `add_node` , `move_node` , `remove_node` , `set_prop` . The envelopes are small:

FROM `packages/contracts/src/index.ts`

```
/** What a peer submits. `opId` makes a resend harmless; `baseSeq` is the last seq the peer
had seen. */
export const ClientOp = z.strictObject({ opId: z.uuid(), baseSeq: z.number().int().min(0),
op: Op });

/** What the room broadcasts: the op, its place in the one true order, and who made it. */
export const SequencedOp = z.object({ seq: z.number().int().min(1), opId: z.uuid(), actor:
Actor, op: Op });
```

Note what the client does *not* send: the actor. The room stamps it from the verified session. A client-supplied actor could claim to be anyone.



1. **Local.** Alice's replica validates the op against the document she sees, applies it at once, and puts it in a `pending` list. She never waits for the network.
2. **Send.** The op goes out with a fresh `opId` and the `baseSeq` she had seen. At most 50 unanswered ops are on the wire at once.
3. **The room decides.** One op at a time, through a queue. Dedupe first, then the `stale` guard, then validation against the design-system manifest, then size limits. An op that would change nothing gets a private `ack` and no number. A real change is made durable first (the seam exists; the journal is epic 6), then applied, then given the next `seq`.
4. **Broadcast.** Everyone gets `{seq, opId, actor, op}`. The sender's copy is her acknowledgement. A refusal goes to the sender only, with a named reason.
5. **Replicas catch up.** Each applies the server's op to its `confirmed` document and rebuilds the optimistic one.

The two documents in every client

A replica holds `confirmed` (what the server said, in server order, never guessed) and `optimistic` (confirmed plus my unanswered ops: what I see). When a remote op arrives, the guess is thrown away and made again:

FROM `packages/peer-client/src/replica.ts`

```
function rebuild(): void {
  revision++;
  optimistic = { ...confirmed, nodes: { ...confirmed.nodes } };
  for (const each of pending) applyOpInto(optimistic, each.op);
}
```

This is `git rebase`: my local commits are replayed on top of the new upstream. It also gives Figma's no-flicker rule for free. My pending "Buy" is re-applied over Bob's confirmed "Pay", so my label does not jump while I wait.

`baseSeq` looks like an optimistic-locking version column, but it is used differently. We do not reject an op because the document moved on: that would turn every concurrent edit into an error. It is only a guard for one case, shown next.

Resends: dedupe, and the rule for what the room forgot

FROM `apps/sync/src/room.ts`

```
const key = `${peer.actor.id}:${opId}`;
const before = remembered.get(key);
if (before) {
  peer.send({ type: "op", ...before.op }); // the ORIGINAL answer, not a second apply
  return;
}
...
if (baseSeq < forgottenUpTo) {
  refuse("stale");
  return;
}
```

The room remembers recent ops (20,000 ops or 8 MB) so a resend gets its original answer. Memory is bounded, so there must be a rule for what falls outside it. If an op was written against a `seq` older than anything the room remembers, the room cannot know whether it already applied it. It says `stale`, and the client resyncs instead of risking a second apply.

Conflicting intent, settled by order

SPEC.md §2.4 fixes the rules. Last writer wins per `(nodeId, key)` by `seq`. A remove beats any concurrent edit under it: the late edit finds no node and is refused as `gone`, silently, because the user did nothing wrong. A move that would create a cycle is refused as `cycle`. An `index` is clamped, and means the node's final position.

In code this is one function, `plan(doc, op)` in `packages/doc-model/src/index.ts`. It is total: any op on any document has a defined result, and an op that no longer makes sense simply changes nothing. "A remove beats an edit" needs no special case.

One difference from Figma: we order children by a plain integer index, not fractional positions. The server's order makes that safe. The cost: if Bob inserts ahead of Alice, her "index 2" may land one slot off from what she saw. For a layout tree we accept that.

Problem by problem, with the bugs that taught us

PROBLEM	OUR MECHANISM	WHERE	WHAT IT COST US
1 Order, 2 Convergence	One room per document gives each op the next <code>seq</code> . A queue handles one op at a time, because <code>persist</code> is awaited.	<code>apps/sync/src/room.ts</code>	Without the queue, two peers could both "successfully" add the same node id while the first write was still awaited.
3 Instant feel	<code>confirmed</code> + <code>pending</code> = <code>optimistic</code> ; <code>rebuild()</code> .	<code>packages/peer-client/src/replica.ts</code>	Copying only the node map: about 1 ms per remote op at 5,000 nodes, against 6.5 ms with a deep clone.
4 Duplicates	Dedupe per <code>(sender, opId)</code> , before validation. A resend keeps its <code>opId</code> and <code>baseSeq</code> .	<code>room.ts</code> , <code>replica.ts</code>	The key must include the sender: a broadcast shows every <code>opId</code> to everyone. And a late answer at or below my <code>seq</code> is not applied again, or it would overwrite a newer value.
4 What the room forgot	<code>stale</code> via <code>baseSeq</code> . Only an op that no earlier connection carried is sent again.	<code>room.ts</code> , <code>replica.ts</code>	A stale resend could put an old write on top of a new one. "Would this op change the document now?" is not "was it applied?".
4 No-op acks	An op that changes nothing: <code>ack</code> to the sender, no <code>seq</code> , no broadcast. The client believes it.	<code>room.ts</code> , <code>replica.ts</code>	A livelock: the room remembered its ack, the replica distrusted it and resynced, the resend got the same ack, for ever. Each feature was correct alone. Found at about 1 seed in 700.
5 Reconnect	Fresh session per connection, a full <code>welcome</code> , pending ops resent. Backoff with jitter, reset only after a connection that lasted.	<code>packages/peer-client/src/peer.ts</code>	Resetting the backoff on every welcome meant a crash-looping server got a reconnect every 250 ms, for ever.
5 Frames during load	Listeners attached before the first <code>await</code> ; early frames are queued until the room has loaded.	<code>apps/sync/src/server.ts</code>	A lost op. A test that failed 1 run in 6 was real: the message listener was attached after an <code>await</code> .
6 Conflicting intent	<code>plan()</code> is total; <code>validate()</code> names the reason (<code>gone</code> , <code>cycle</code> , ...).	<code>packages/doc-model/src</code>	<code>__proto__</code> as a prop name, and <code>-0</code> across a JSON hop, each made sender and peers diverge. Both are refused in the contract.
7 Validity, identity	Every op is validated against a manifest generated from the design system. The room stamps the actor.	<code>packages/contracts</code> , <code>doc-model/src/validate.ts</code>	Two of our own limits disagreed (a 500 KB contract, a 64 KB frame): a valid op would have killed the socket and been resent for ever.

PROBLEM	OUR MECHANISM	WHERE	WHAT IT COST US
8 Overload	Token bucket per actor (user, agent run, git), charged on arrival. After a refusal, later ops are refused too, to keep order. The client restarts at one op in flight.	<code>room.ts</code> , <code>replica.ts</code>	Resending the whole window after "too fast" cost 18,524 refusals for 600 ops.
9 Durability (today)	The whole document is saved when the last peer leaves and on shutdown. Save first, then close sockets.	<code>apps/sync/src/server.ts</code>	"Terminate sockets, then await pending saves" lost everything: the close handlers that start the saves run on a later tick.

How we know it converges

`apps/sync/src/sim.ts` is a seeded simulator. It drives the real room and the real replica with three peers, under delay, reordering across peers, dropped connections and room reloads. One seed, one run, byte for byte, so any failure can be replayed.

After every step it checks invariants: no `opId` at two seqs, no gap in `seq` , last writer wins (computed apart from the code under test), the room equals a replay of its own log, a caught-up replica equals the room. At the end everyone must converge with nothing pending. The drain is bounded, so a livelock is a failure, not a hang.

Two lessons: random ops rarely collide, so generate conflicts on purpose; and seeds 1 to 40 never found the livelock, so sweep thousands of seeds once and commit the ones that fail.

Planned, not built yet

Today's honest limit. The `persist` seam is in the room, but no journal is wired to it. A `kill -9` on the sync server loses the edits made since the room opened. The code says so itself.

Epic 6: journal and snapshots. Every op is appended to a Postgres table *before* it is broadcast. Snapshots go to MinIO. Opening a document loads the newest snapshot and replays the rows after it. This is event sourcing. A unique index on the op id then makes dedupe permanent, and the `stale` case goes away.

Epic 7: more than one sync node. A Redis lease decides which node owns a room. A lease alone is not safe. `learning-tests/redis/test.ts` (finding 4) blocked a process for 2 s on a 300 ms lease: another process took the lease about 1.7 s before the first one woke up, still believing it was the owner. So each lease comes with a **fencing token**, a number that only grows, and the journal append checks it in the same SQL statement. A zombie's write is refused by the database, whatever the zombie believes.

5. The decision framework

Ask these six questions. The answers pick the family.

QUESTION	IF YES	IF NO	OUR ANSWER
Is a server always in the path?	Let it decide order (C or E)	You need a CRDT (D)	Yes. Peers get a session from the API before they connect.
Must edits work offline for days, or peer-to-peer?	CRDT (D)	C or E is enough	No. A short disconnect is covered by pending ops.
Is the data free text?	OT, or a text CRDT	Objects and props: E with LWW is enough	No. A tree of typed components and props.
Must someone enforce schema, permissions, rate limits, audit?	You need an authority: C or E	D becomes attractive	Yes, all four. This decided it.
Many writers on one document at once?	C, D or E. Not A or B	A lock or a version column may do	Yes: people, plus an AI agent making many ops.
Can users accept "converged but surprising"?	Automatic merge rules are fine	Prefer explicit refusals with reasons	Mostly no. A refused op names its reason.

The sentence to say

"There are five real options: locking, whole-document last-write-wins, Operational Transformation, CRDTs, and a server-ordered op log.

We picked the server-ordered op log, because for us the important part is that **every edit, from a person, the AI agent or git, passes one gate**: validated against the design-system manifest, attributed by the server, rate-limited and auditable. And a central server already exists.

CRDTs do not give you that gate for free, because no one is the authority. OT does, but it is needlessly complex for a tree of typed props: we have no text positions to transform. Locking and whole-document LWW are not multiplayer."

What we gave up

- **Offline-first and peer-to-peer.** Without the server there is no order, so no confirmed edits.
- **One room is one writer.** A single document cannot use more than one process. We scale by sharding documents across nodes, not by splitting a document.
- **The server is on the critical path.** When storage is down, the room must refuse ops (`unavailable`) rather than accept what it cannot keep.

- **No merging inside a value.** Two people typing in one text prop: one wins. The same limit Figma states.

When I would choose differently

- **A CRDT** for a notes app that must work on a plane for a week, for peer-to-peer sync, or when I cannot run a server per document. Also when a library like Yjs gives collaborative text out of the box and nothing needs enforcing.
- **OT, or a text CRDT inside the server-ordered design**, if rich text editing by several people in one paragraph becomes a core feature.
- **A lock** for rare, heavy, non-mergeable edits: publishing a release, editing a binary asset.

6. Interview crib

Why not Yjs?

Yjs solves merging without an authority. We have an authority and we need it: every op must be validated against the manifest, attributed and rate-limited. With a server ordering ops, convergence is free, so the CRDT machinery would be cost without benefit. I would reach for it if offline-first became a requirement.

What happens to an op in flight when the socket drops?

It stays in the client's `pending` list, still visible to the user. The client reconnects with backoff and jitter, gets a fresh `welcome`, rebuilds, and resends everything pending with the same `opId` and `baseSeq`. If the room had applied it, the room answers with the original sequenced op.

How do you avoid applying a resend twice?

The room dedupes on `(sender, opId)` before validating. Its memory is bounded, so an op older than that memory is refused as `stale` and the client resyncs. The planned journal has a unique index on the op id, which makes this permanent.

What if two servers both think they own the room?

That is planned work (epic 7). A lease cannot prevent it: a paused process does not know its lease expired, and our Redis learning test measured exactly that. The fix is a fencing token checked by the storage layer on every journal append, so the stale owner's write fails in Postgres.

How do you know it converges?

A seeded simulator runs the real room and real replica under delay, reorder, drops and reloads, and checks invariants after every step. It found a real livelock at about 1 seed in 700. The oracle for last-writer-wins shares no code with the model it judges.

How does one document scale?

It does not need to: a room is one document's editors, and a person dragging makes about 60 ops a second. The system scales by placing different documents on different nodes. The session endpoint returns the socket URL, so routing was a hook from day one.

How is the AI agent different from a human editor?

It is not, and that is the design. It edits through the same `peer-client`, as actor kind `agent` with its own run id and its own rate budget. It waits on a per-op `settled` outcome, so each tool call gets a real answer. The git peer (planned) will use the same path.

Where does Raft fit?

Under the server, not between the browsers. It replicates a server's log across machines. We plan to get the same safety from Postgres as the single journal plus a fenced lease, which is less to operate.

7. Glossary

<code>op</code>	One small change, such as <code>set_prop</code> . We send ops, never whole documents.
<code>seq</code>	The number the room gives an accepted op. The one true order. Like a Kafka offset.
<code>baseSeq</code>	The last <code>seq</code> the sender had seen when it made the op. Lets the room say "too old to judge" (<code>stale</code>).
<code>confirmed</code>	The document as the server has stated it. Never guessed.
<code>optimistic</code>	Confirmed plus my unanswered ops. What I see on screen.
<code>idempotent</code>	Doing it twice has the same effect as once. Our <code>opId</code> makes a resend idempotent.
<code>LWW</code>	Last writer wins. The honest question is always "last by whose order?". Ours: the room's <code>seq</code> .
<code>tombstone</code>	A marker a CRDT keeps for a deleted item, so late edits can still be placed. A source of growth.
<code>fencing token</code>	A number that only grows, issued with a lease. Storage refuses writes that carry an older one.

8. Sources

- Evan Wallace, "How Figma's multiplayer technology works", Figma blog, 16 October 2019. figma.com/blog/how-figmas-multiplayer-technology-works. Read for this page; the three quotes are exact.
- Evan Wallace, "Realtime editing of ordered sequences", Figma blog, 2017 (fractional indexing). figma.com/blog/realtime-editing-of-ordered-sequences. Read for this page.
- Kleppmann, Mulligan, Gomes, Beresford, "A highly-available move operation for replicated trees", 2021. martin.kleppmann.com/papers/move-op.pdf
- Martin Kleppmann, "How to do distributed locking", 2016 (why a lock needs a fencing token). martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html
- Yjs: docs.yjs.dev. Automerge: automerge.org. Raft: raft.github.io.
- This repository: `SPEC.md` (§2, §2a, §4, epics 6 and 7), `HANDOFF.md` §7, the files named above, and the project's `bd memories` for E2.2, E2.3, E2.4, E2.8 and E2.9.

Noon MVP handbook · written from the code at commit `edbd19b`. Everything under "planned" is design, not code.